



Domino drift effect approach for probability estimation of feature drift in high-dimensional data

Gábor Szűcs¹ · Marcell Németh¹

Received: 9 May 2023 / Revised: 17 January 2025 / Accepted: 26 January 2025 /
Published online: 13 February 2025
© The Author(s) 2025

Abstract

Concept drift (and data drift) is a common phenomenon in machine learning models, where the statistical properties of the input data change over time, leading to a decrease in model performance. Detecting data drift is crucial for maintaining the accuracy and reliability of machine learning models in real-world applications. While previous data drift detector approaches can identify if a drift has occurred, these approaches cannot localize which specific features have caused the drift. Feature drift detectors solve this deficiency, but the required number of detectors is equal to the number of dimensions, which is a resource-intensive solution in high-dimensional data. In this paper, we propose a novel approach for feature drift analysis and drift detection based on a domino effect caused by the correlation of features. Our approach, the so-called Domino drift effect (DDE), is based on the empirically proven assumption that an initial reference correlation can be utilized as a proxy for detecting other drifting features. The method analyzes the correlating and drifting behavior, and by using only a subset of all features, it derives inference about the drifting of the remaining features, if co-drifting phenomena occur in the data stream. At co-drifting phenomena, the DDE method can estimate the probability of feature drift, which is particularly useful in high-dimensional datasets. To evaluate the effectiveness of our approach, we conducted experiments on four real-world datasets. The results show that our approach can effectively be used to predict feature drift in the whole dataset, and it has potential industrial applications.

Keywords Data streams · Concept drift · Feature correlation · Feature drift detector · High-dimensional data · Probability design

1 Introduction

In data stream analysis, researchers and developers build models to capture information hidden in the data, either for description or prediction [1]. Methods for data stream analysis can be used in lots of potential application areas including financial time series analysis [2],

✉ Gábor Szűcs
szucs@tmit.bme.hu

Marcell Németh
nemethm@tmit.bme.hu

¹ Department of Telecommunications and Artificial Intelligence, Budapest University of Technology and Economics, Műegyetem rkp. 3., Budapest 1111, Hungary

industrial process monitoring, social media activity, or customer behavior data capturing. Standard machine learning techniques are widely employed for these data streams [3], but they are inadequate for the evolving nature of continuous changes in the non-stationary environment [4], e.g., when customers' behavior is changing in credit card fraud detection [5]. Another example is the change in mobile usage; there has been a surge in interest in mobile data stream mining, aiming to construct near real-time models for data stream mining applications that run on mobile devices [6].

This kind of problem is often tackled by discarding past knowledge, despite its potential relevance in the case of recurrent concepts. The best solutions contain a concept drift detection method [7], and based on the result of the detector, they use only the new concept after the drift and they use the past knowledge until the previous concept.

Finding changes in multidimensional unlabeled data is still another kind of challenge. For a large number of dimensions, applying the PCA (principal component analysis) is a natural method for feature extraction prior to the change detection [8]. The authors investigated not only the data drift but feature drift as well, where their detector localized which features have drifted via conditional distribution tests [9]. In the literature, there are lots of drift detectors, like K-L distance and Hotelling's t^2 tests that are able to detect the changes in univariate or multivariate data streams [10]. The complex drift detectors contain an algorithm instead of a simple hypothesis test; these algorithms use a supervised or unsupervised approach. For example, margin density drift detection (MD3) algorithm [11] belongs to an unsupervised one; this tracks the number of samples in the uncertainty region of a classifier, as a metric to detect drift.

This paper is concerned with data drift, where multidimensional data consist of lots of features (the number of features is equal to the number of dimensions in the multidimensional dataset). Our research focused on feature drifts and relationships among them. The definition of feature drift (between t and $t + 1$) with i th feature can be seen in the next equation, i.e., the data belong to two different distributions.

$$P_t(f_i) \neq P_{t+1}(f_i) \quad (1)$$

An interpretable drift detection method should also locate the source of the drift and characterize its nature [12]. While previous papers investigated the univariate or multivariate data drifts, none of them examined the relationships among the drifting features. The aim of our research was to investigate these relationships, particularly the relationship between the probability of feature drift and its correlation with other features. The further aim was to explore the possibility of the usage of the co-drifting phenomenon, i.e., if there is a strong connection between the drifting probability and the correlation inside the feature set, then this phenomenon can be used in the future for the prediction task. In the high-dimensional dataset, the number of features is large; in this problem, drift detectors for all features require a lot of resources. Our research is concerned with this problem, i.e., how only a few detectors can solve the drift prediction task for all features.

The main contribution of this paper is the data drift analysis in high dimensions, and it is the first to investigate the co-drifting phenomenon. Our contributions are as follows:

1. Introduction of a new concept, the Domino drift effect (DDE), for the co-drifting phenomenon, which is based on the assumption that the strong correlation between features is proportional to the probability of occurrence of the double drift phenomenon.
2. A method using feature correlation as a drift proxy, which can be used to assign probability values to the expected drift of the remaining features by analyzing only a subset of all features.

3. Calculations and predictions about the drifting chances of one of the feature members of the pair without actually performing a drift detection on it, purely based on the drift behavior of the other feature.
4. Design of the DDE probability lift curve, which can show whether the drift affects the individual features independently or affects groups of features based on the nature of the curve's trend. We created an algorithm for the DDE method, and the implementation is published on GitHub with the corresponding datasets as well: <https://github.com/marcell-nemeth/DominoDriftEffect>.
5. Experiments on four real-world datasets with low and high dimensions, containing both artificially induced and naturally occurring data drifts.

The rest of the paper is organized as follows. In Sect. 2, we present the proposed approach with the co-drifting phenomenon. Section 3 contains the literature review. Section 4 contains the application of our method with the evaluation plan. In Sect. 5, we present the experimental results with four datasets. Section 6 consists of the conclusion about the proposed method.

2 Proposed approach

2.1 Notations for Domino drift effect (DDE)

The probability that a feature's drift will also happen together with another feature's drift whose relationship with previous feature (measured in the reference window) is stronger/greater (or equal) than a threshold can be determined with a sufficiently high number of drifts. This relationship is measured by the absolute value of the Pearson linear correlation coefficient (ρ_{ij}), and this threshold is denoted by z . Let's call this phenomenon the Domino drift effect (DDE), and its probability is expressed by the formula below $\text{DDE}(z) := \Pr(\text{DDE} \mid z)$ where $0 \leq z \leq 1$ and

$$\Pr(\text{DDE} \mid z) = \Pr(P_t(f_j) \neq P_{t+1}(f_j) \mid P_t(f_i) \neq P_{t+1}(f_i), |\rho_{ij}| \geq z) \quad (2)$$

As we do not have y labels, we can only measure feature drift (data drift) in an unsupervised way. Let S_{feat} be the set of all features and denote their number by m :

$$m = |S_{feat}| \quad (3)$$

Furthermore, let's take the set of features where any drift occurred, denote this by S_{drift} and the size of this set by k :

$$S_{drift} = \{j \mid P_t(f_j) \neq P_{t+1}(f_j)\} \quad (4)$$

$$k = |S_{drift}|. \quad (5)$$

Let us denote the number of drifting j features by n_i for which the absolute value of linear correlation coefficient with f_i is greater (or equal) than a threshold.

$$n_i(z) = |\{j \mid j \in S_{drift}, |\rho_{ij}| \geq z\}|, \quad 0 \leq z \leq 1 \quad (6)$$

Let us denote the number of all j features by N_i for which the absolute value of the linear correlation coefficient with f_i is greater (or equal) than a threshold.

$$N_i(z) = |\{j \mid |\rho_{ij}| \geq z\}|, \quad 0 \leq z \leq 1 \quad (7)$$

Having these notations introduced, the aim for the following is to measure the probability of DDE. The next subchapters present the theoretical design of the DDE model and the probability lift curve.

2.2 DDE probability lift curve

The value of n_i can show high variance depending on the value of i ; however, $\frac{n_i}{N_i}$ usually results in the same ratio for different i values. In the following, we assume that for a given data set this ratio is approximately equal for two arbitrarily chosen features.

$$\frac{n_i(z)}{N_i(z)} \approx \frac{n_j(z)}{N_j(z)} \quad \forall z, \forall i, j \in S_{feat} \quad (8)$$

Even though this assumption is not a strong limitation, it helps a lot in the upcoming calculations, which is why this simplification is utilized in our DDE model. The $n_i(z)$ gives the number of drifting j features, where $|\rho_{ij}| \geq z$, and we can calculate the average number of drifting j features from arbitrary drifting feature i ; this $n_i(z)$ average is denoted by $n(z)$.

$$n(z) = \frac{\sum_{i=1}^k n_i(z)}{k} \quad (9)$$

In a similar way, the average number of all j features from arbitrary drifting feature i where $|\rho_{ij}| \geq z$ can be also calculated; this $N_i(z)$ average is denoted by $N(z)$.

$$N(z) = \frac{\sum_{i=1}^k N_i(z)}{k} \quad (10)$$

Since $\frac{n_i(z)}{N_i(z)}$ results in approximately the same ratio for different i values, they will not only be equal to each other but also to the ratio of the two averages, i.e., $n(z)$ divided by $N(z)$.

$$\frac{n_i(z)}{N_i(z)} = \frac{n(z)}{N(z)} \quad \forall z, \forall i \in S_{feat} \quad (11)$$

To calculate the probability of the Domino drift effect (DDE), let's take each drifting feature (all of them have a probability of $\frac{1}{k}$) and, starting from this, calculate the probability that its neighbor is also drifting (by neighbor we mean those features for which $|\rho_{ij}| \geq z$), then these should be summarized.

$$\Pr(DDE | z) = \sum_{i=1}^k \frac{1}{k} \frac{n_i(z)}{N_i(z)} = \frac{n(z)}{N(z)} \quad (12)$$

This ratio will be denoted by $r_{n/N}$, giving the DDE probability for different z values.

$$r_{n/N}(z) = \frac{n(z)}{N(z)} \quad (13)$$

Both $n(z)$ and $N(z)$ values are the results of an average calculation. Let's find another option for calculating the probability of DDE without considering each feature individually and then average them. Taking the feature pairs where the correlation is stronger (or equal to) z and within that, those with both features drifting, we can also examine their ratio. The former is denoted by $B(z)$, the latter by $b(z)$, and then, their proportion ($r_{b/B}$) can be formed.

$$B(z) = |\{(i, j) \mid |\rho_{ij}| \geq z\}| \quad (14)$$

$$b(z) = |\{(i, j) \mid i, j \in S_{drift}, |\rho_{ij}| \geq z\}| \quad (15)$$

$$r_{b/B}(z) = \frac{b(z)}{B(z)}. \quad (16)$$

Let's examine the relationship between $r_{b/B}$ and $r_{n/N}$. It follows from the definitions that the below equalities stand as

$$\sum_{i=1}^k n_i(z) = 2b(z) \quad (17)$$

$$\sum_{i=1}^k N_i(z) + \sum_{i=k+1}^m N_i(z) = 2B(z), \quad (18)$$

where the first term contains the summation of drifting features, while the second term contains the summation of non-drifting features (since every pair was considered twice, there is a multiplier of 2 on the right).

$$\begin{aligned} r_{b/B}(z) &= \frac{b(z)}{B(z)} \\ &= \frac{\sum_{i=1}^k n_i(z)}{\sum_{i=1}^k N_i(z) + \sum_{i=k+1}^m N_i(z)} = \frac{kn(z)}{kN(z) + (m-k)N(z)} = \frac{kn(z)}{mN(z)} \end{aligned} \quad (19)$$

$$r_{b/B}(z) = \frac{k}{m} r_{n/N}(z) \quad (20)$$

As can be seen from the formula, $r_{b/B}(z)$ can be clearly determined from the value of $r_{n/N}(z)$ by multiplying the ratio $\frac{k}{m}$, which is the ratio of drifting features. The advantage of our model is that no matter how discrete the distribution of n_i is at different features (it may be a small value for one or a large value for another), we still have the possibility to calculate the probability of DDE through the feature pairs; plotting this on a diagram as a function of z , we get the DDE probability curve.

2.3 Theoretical design of the DDE model

In most cases, the specific value of the DDE probability curve is not interesting but the trend of the curve (e.g., horizontal or increasing). It is worth normalizing the curve by dividing the value taken at point 0, to obtain the DDE probability lift curve (it starts from 1 at zero).

$$r_{n/N}^{(lift)}(z) = \frac{r_{n/N}(z)}{r_{n/N}(0)} \quad (21)$$

Since all pairs must be taken into account for the $r_{b/B}(0)$ value, the ratio of the drifting and all pairs was included in the denominator, therefore

$$r_{b/B}^{(lift)}(z) = \frac{r_{b/B}(z)}{r_{b/B}(0)} \quad (22)$$

$$\frac{r_{b/B}(z)}{\frac{k(k-1)/2}{m(m-1)/2}} = \frac{r_{b/B}(z) m(m-1)}{k(k-1)} \quad (23)$$

Furthermore, the two probability curves ($r_{b/B}(z)$ and $r_{n/N}(z)$) only differ by a constant multiplication, after normalization the two lift curves will be the same:

$$r_{b/B}^{(lift)}(z) = r_{n/N}^{(lift)}(z) \quad (24)$$

The DDE probability lift curve shows whether, in the case of a stronger correlation, the probability that the neighbor of a drifting feature also drifting is higher. If this curve follows a horizontal trend, therefore regardless of the correlation, we always experience the same probability. On the other hand, if it is an increasing curve, then the drift does not only affect the individual features independently but rather affects groups of features between which the relationship is stronger.

3 Related works

Change detection [13, 14] (or drift detection) refers to the methodology that helps determine and identify a time instant when a change arises in the properties of the target object. Concept drift detection methods can be divided into supervised and unsupervised approaches. Supervised methods utilize labeled data to monitor and detect changes by tracking prediction errors or other performance metrics. Unsupervised methods operate without labeled data, focusing instead on the inherent characteristics of the data distributions to detect changes over time.

The unsupervised approach is a more difficult task, thus some solutions try to form a bridge between the two approaches so that the unsupervised can be led back to the supervised one. An unsupervised task can be solved using different conversion techniques, where a dummy classifier is constructed to detect concept drift by monitoring classification outputs [15–17]. For example, unsupervised concept drift detection can be solved using a student–teacher learning approach [18], where an auxiliary model (student) mimics the behavior of the main model (teacher). In controlled experiments, this approach detects concept drift effectively; it signals a small number of false alarms, but requires more time to detect changes. Another solution to this conversion is the Probabilistic Concept Drift Detection (PCDD) [19]. If labeling samples in real-time streaming is not feasible due to resource utilization and time constraints, PCDD relies on the data stream classification process and provides concept drift without labeled samples. In our article, we used only the unsupervised approach, and it was not converted back to the supervised one.

There is no single drift detector that works better than all the others in all scenarios [20]; therefore, we investigated and used four detectors in our work: Jensen–Shannon [21] and ADWIN [22], Wasserstein [23], and Kolmogorov–Smirnov [24, 25] detectors [26]. Many concept drift detectors rely on the Jensen–Shannon detector, e.g., CDJD [27] or CDDDE [28]; the latter one uses Jensen–Shannon divergence with extension of exponentially weighted moving average. Kolmogorov–Smirnov is used in lots of unsupervised concept drift detectors without modifications [29, 30], or with amendment to extend the basic functionality [31]. Wasserstein distance is also an appropriate basic for concept drift detection [32]; although there are some advanced versions (e.g., Wasserstein distance learned feature representations—WDLFR [33]), we used the basic method.

Only a small portion of the papers investigated the correlation as a possibility in data drift detection in machine learning models. These methods utilize the correlation between the features in the input data to identify changes in their statistical properties. The basic idea is that changes in the correlation structure of the features can indicate changes in the data distribution. Similarly, Huang et al. [34] proposed a method based on not only higher-order statistical dependencies between the input features but also utilizing histori-

Table 1 Comparison of the papers that uses correlation analysis for concept drift

References	Unsupervised	Prediction for the next drift	Feature level analysis in a drift	Comp. Resource oriented
[34]	Yes	Yes	No	No
[35]	No	No	No	No
[36]	No	Yes	No	No
[37]	Yes	No	No	No
[38]	Yes	No	No	No
[39]	Yes	Yes	No	No
Our paper	Yes	Yes	Yes	Yes

cal drift trends to estimate the probability of expecting a drift at different points across the stream.

Similarly to correlation-based methods, the covariate shift can also be used in drift detection [35–37]. These methods measure the difference between the joint distribution of the input data and the joint distribution of the target variable and the input data. If the difference is significant, concept drift is detected. However, this method requires knowledge of the target variable, which is not always available in real-world applications.

Dynamic temporal dependencies between labels can be exploited using a label influence ranking method, which leverages a data fusion algorithm and uses the produced ranking to detect concept drift. Label dependency drift detector (LD3) [38], an unsupervised concept drift detector (in the multi-label classification problem area), uses these label dependencies within the data for multi-label data streams.

There is a similar work [39] to our paper, in which concept drift correlations are investigated. That work used Granger causality to determine linear and nonlinear correlations between concept drifts; then, authors used different techniques to find concept drift correlations. This correlation is a causality relationship, where the authors were interested in whether there was a causal relationship between two drifts that were different in time. A concept drift is represented by a snippet of the time series within which the drift is found, and the concept drift correlation is retrieved by the Granger causality definition. With Granger causality, it is not possible to examine simultaneous events, and we had to investigate the relationship among the features at the same time, because our focus was on features inside a drift situation. Another difference is that other papers dealt with concept drift in the multidimensional level, but our analysis was in the feature level.

Table 1 presents the comparison of the papers that uses correlation analysis in their concept drift methodology. Some of the unsupervised methods deal with prediction for the next drift, and none of them contains feature level analysis in a drift, except our paper. Furthermore, our research was computational resource-oriented work, so not only the goodness indicators were important but the performance (running time) as well.

All these papers investigated the correlation among the drift or the features, but not among the drifting features. The previous works focused on drift detection, so the correlation was input information for these detectors. In this paper, we explored the relationship between the probability of feature drift and its correlation with other features, which is a different aim from the previous papers, because we are interested in the co-drifting phenomenon (this phenomenon describes the feature structure of the drift).

Table 2 Used datasets with main attributes

Dataset	No. Features	No. Classes	Drift source	Drift type
Covertypes [40]	16	2	Artificially induced	Abrupt/incremental (synthetic)
CICIDS [41]	19	2	Natural	Abrupt
Insects [42]	200	3	Natural	Abrupt/incremental
Heartbeats [43]	280	4	Natural	Abrupt

4 Prediction by the DDE method

4.1 Used datasets in the experiments

In this chapter, the usage of the applied DDE method is presented, putting the focus on the two main, drift curve fitting and prediction phases. During the research, we examined two lower-dimensional (CICIDS with 19 features and Covertypes with top 16 features) and two high-dimensional datasets with the below-mentioned parameters (Insects with 200 features and Heartbeats with 280 features) in Table 2.

4.2 Detecting drift points

However, there are many benchmark datasets in the drift detection literature; it can generally be said that drift change points are typically not clearly defined and labeled. Except for Covertypes, where the drifts were artificially induced by changing the values of the features, the other datasets examined in our paper naturally contain drifts.

For all datasets, drift detection was achieved by using univariate, statistical methods. Four of the most popular, Jensen–Shannon (JS) [21], ADWIN [26], Wasserstein [23], and Kolmogorov–Smirnov [24] detectors were utilized to monitor changes between the distributions of two windows (reference and analysis windows) for each feature. Our aim was to use datasets that have at least two drift points to be able to utilize the DDE approach: one to fit the DDE curve and one to predict.

Detected drift points on the CICIDS dataset (as an example) via JS distance are shown in Fig. 1, where the drift points are marked with red, dashed lines, and different lines in the figure correspond to different features in the dataset. The horizontal axis shows the time steps, and the JS distance can be seen on the vertical axis. JS distance values are calculated by comparing the empirical distributions of consecutive data windows. Peaks in JS values indicate greater differences between two windows, therefore indicating drifts. In the Experimental Results chapter, different signaled drift points are evaluated to show the behavior of the DDE method-based drift likelihood prediction.

4.3 Drift curve fitting and prediction algorithm by DDE

In the following, the algorithm utilizing the DDE method will be detailed to fit an initial drift curve and predict DDE scores for future streams.

The whole procedure with drift detection and the details of our Domino drift effect (DDE) prediction algorithm is given in Algorithm 1. Let $S_{check} \subseteq S_{feat}$ be the set of features on which direct drift detection can be performed, these are the monitored features. Let's denote

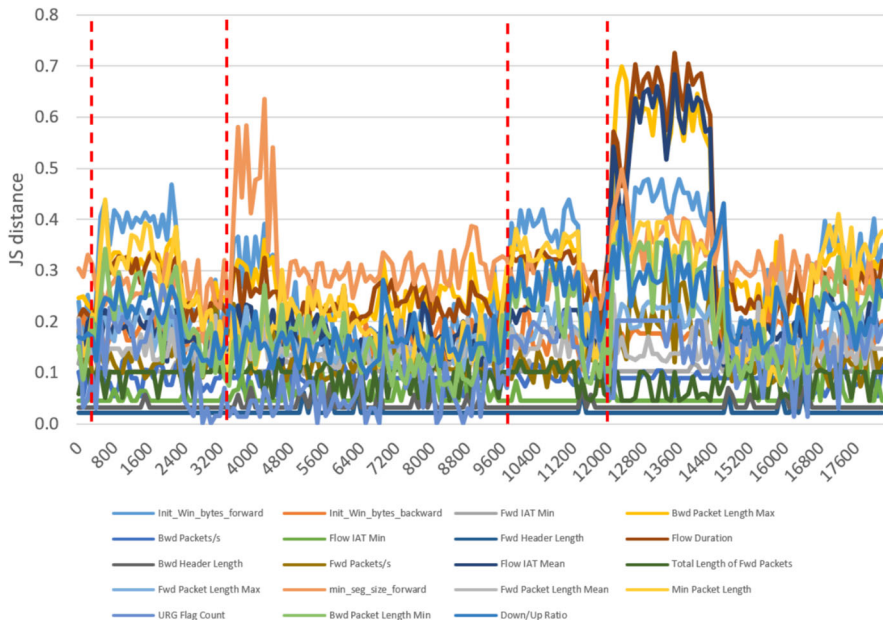


Fig. 1 Detected drift points on the CICIDS dataset via JS distance. Solid lines are features; drift points are marked with red, dashed lines

the subset of the dataset by $D_{S_{check}}$ containing all features in S_{check} and a slice of $D_{S_{check}}$ at time t as $D_{S_{check},t}$. For drift detection, set the size of the reference window W_{ref} to r and the analysis window W_{an} to q . Using the before-mentioned notations, drift detection will be performed by comparing time windows $W_{ref} = \{D_{S_{check},t} \dots D_{S_{check},t+r}\}$ and $W_{an} = \{D_{S_{check},t+r+1} \dots D_{S_{check},t+r+q+1}\}$ of $D_{S_{check}}$, where t is a running index. As a rule of thumb, r and q should be equal to achieve a statistically fair comparison.

Next, drift detection should be performed on every feature $f_i \in S_{check}$, $i = 1, 2, \dots, |S_{check}|$ and denote the set of drifting features as $S_{drift} \subseteq S_{check}$, where $|S_{drift}| = k$. As the next step (if at least one feature triggers a drift), all possible feature pairs S_{pairs} should be created as a Cartesian product of S_{drift} and $S_{rest} = S_{feat} \setminus S_{check}$. For all feature pairs, the Pearson linear correlation coefficient and the double drift occurrences should be calculated. Based on these values, the DDE probability curve can be constructed (curve fitting phase) using the equations in Chapter 2.2, which could be utilized to predict future drift probabilities in the unmonitored features (S_{rest}).

This calculation must be performed at least once to obtain a DDE probability curve and is recommended to repeat regularly to refine every time a drift is detected. Once a DDE probability curve is constructed, the approximation (prediction phase) of the drift score of an unmonitored feature f_i can be performed via aggregating DDE probability values from all pairs of which the specific feature is a member:

$$DDE_{score}(f_i) = \frac{\sum_{j=1}^{L_i} DDE(\rho_{ij})}{L_i} \quad (25)$$

where L_i scaling factor is equal to the number of pairs in S_{pairs} where f_i can be found and ρ_{ij} is the Pearson linear correlation coefficient between f_i and f_j features.

The correlation between features is calculated by the algorithm not only once at the beginning and not at each step, but after each drift detection. This allows the method to track the change in correlations to some extent on the one hand, and on the other hand, it is not as wasteful of resources as calculating these values at each step.

Summarizing the process detailed above, our method necessitates an initialization phase during which each feature is monitored by a drift detector until the first drift is detected. The resulting data from this initial drift are used to construct a curve, which then facilitates the prediction of subsequent drifts. Additionally, averaging the results from multiple drifts can enhance the accuracy of the curve. A limitation of this approach is its reliance on detecting at least one initial drift; without this initialization phase, it is not possible to construct the curve or make subsequent predictions, as all machine learning and statistics-based methods depend to some extent on historical information. Our assumption is the same that is the internal structure of across multiple features (the correlation of them) is similar with the past.

Algorithm 1 Drift detection and Domino drift effect (DDE) prediction

Require:

S_{feat} —Set of all features in the dataset
 $D_{S_{feat},t}$ —Dataset slice at time t
 r, q —Sizes of reference and analysis windows
 z —Threshold for Pearson correlation coefficient
 T —Total time steps

Output:

$DDE_{score}(f_i)$ —Predicted drift score for unmonitored feature f_i

1: Initialization:

```

2: Set monitored features  $S_{check} \subseteq S_{feat}$ 
3: Set unmonitored features  $S_{rest} = S_{feat} \setminus S_{check}$ 
4: Create empty set  $S_{drift}$ 
5: for  $t = 1$  to  $T - \max(r, q)$  do
6:   Define  $W_{ref} = \{D_{S_{check},t}, \dots, D_{S_{check},t+r}\}$ 
7:   Define  $W_{an} = \{D_{S_{check},t+r+1}, \dots, D_{S_{check},t+r+q+1}\}$ 
8:   for Each feature  $f_i \in S_{check}$  do
9:     Compute drift using Jensen–Shannon / Kolmogorov–Smirnov / Wasserstein distance based or
       ADWIN detectors
10:    if Drift detected for  $f_i$  then
11:      Add  $f_i$  to  $S_{drift}$ 
12:    end if
13:  end for
14: end for
15: Construct Feature Pairs:
16: Create Cartesian product  $S_{pairs} = S_{drift} \times S_{rest}$ 
17: for Each pair  $(f_i, f_j) \in S_{pairs}$  do
18:   Compute Pearson correlation coefficient  $\rho_{ij}$ 
19:   Compute double drift occurrence probabilities
20: end for
21: Construct DDE Probability Curve:
22: Fit  $DDE(z)$  curve using  $n_i(z)$  and  $N_i(z)$  for all  $f_i \in S_{drift}$ 
23: Drift Score Prediction:
24: for Each  $f_i \in S_{rest}$  do
25:    $L_i \leftarrow$  Number of pairs in  $S_{pairs}$  where  $f_i$  appears
26:   Compute  $DDE_{score}(f_i)$  using Eq. 25
27: end for
28: Repeat:
29: Regularly update  $DDE(z)$  curve whenever new drifts are detected.
  
```

4.4 Further use cases

We can examine our DDE model for synthetic drift generation. Let's assume that we have a real data set with m features, where the drift is (synthetically) generated by randomly taking k features and then manipulating them until drifting. Since we did not consider the correlation between the features, the probability of starting from a drifting f_i , another f_j also drift will be $\frac{k-1}{m-1}$ independent of z . This will result in a horizontal DDE probability curve and a horizontal DDE probability lift curve.

Furthermore, correlation is a proxy for mutually drifting features; therefore, our method allows for the efficient use of resources in detecting and correcting drifting features, resulting in increased accuracy and efficiency in data analysis. This approach may find applications in various fields where checking each feature would be expensive. By identifying the correlation between features, this method can detect when one feature is likely to drift based on changes in another correlated feature, which may not be detected by univariate methods. Therefore, the method based on correlation can complement univariate drift detection methods and provide additional insight into the behavior of multiple features in a dataset.

In the following, a potential industrial application will be detailed through an easy-to-interpret example. Let us suppose that the goal is to monitor the terrain sensitivity of a self-driving vehicle. A digitally enhanced car typically has thousands of sensors, each of those contributing to describing the current state of the vehicle. For example, multiple vibration sensors are physically placed close to each other, therefore showing similar responses to terrain changes. Let us suppose that the driving assist model loses accuracy after leaving the highway to a countryside road and would like to explore the reason. We suspect the underlying phenomena should be related to the quality of the roads, therefore starting to analyze the corresponding data coming from the vibration sensors.

Having finished analyzing the time series of the first sensor, we detect significant changes in the vibration amplitudes due to the terrain quality changes and record it as a data/concept drift contributing to the decreased model accuracy. However, so far, there is only finished running univariate drift detection method on the first sensor feature data; the DDE method allows determining whether running the same tests on the other sensory features would be truly necessary due to the high initial correlation or would be unnecessary because of the drift probability is highly unlikely (because of the uncorrelated nature).

5 Experimental results

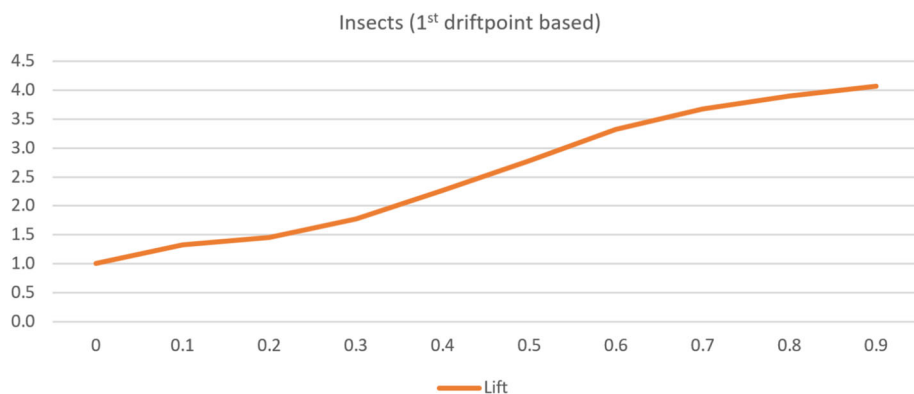
This chapter details the quantitative measurements of the DDE method on four datasets containing different types of drifts. In the first subsection, we present a lift curve example in a high-dimensional dataset as a results of the initialization phase.

5.1 Lift curve example as a results of the initialization phase

To obtain DDE probability and lift values, the number of overall and double drifting feature pairs should be calculated on different correlation threshold levels. We selected only a high-dimensional dataset for demonstrating a typical result of theoretical 3.2 and 3.3 subsections. The results are based on feature drifts in the initialization, i.e., on the first drift. Table 3 shows the before-mentioned pair numbers and the corresponding probability and lift curves (Fig. 2) on the first drift point of the Insects dataset. The lift values in the last column of Table 3

Table 3 Number of feature pairs on different correlation thresholds with the corresponding DDE probability and lift values on the 1st drift of the Insects dataset

Threshold	No. Feature pairs	No. Double drifting feature pairs	DDE probability	DDE lift
0.0	19,900	1176	0.24	1.00
0.1	13,755	1078	0.32	1.33
0.2	8401	722	0.35	1.45
0.3	5532	581	0.43	1.78
0.4	3332	446	0.55	2.27
0.5	2158	354	0.67	2.78
0.6	1501	295	0.80	3.33
0.7	1105	240	0.89	3.68
0.8	799	184	0.94	3.90
0.9	491	118	0.98	4.07

**Fig. 2** Lift curve on the 1st drift of the Insects dataset

(and the curve in Fig. 2) come from Eq 20 and 21 (it can be seen at the end of our derivation in Eq 24 that they are the same). These lift curves were also constructed for the other three datasets, from which we could derive insights into drift characteristics based on the curve trends detailed in the next sections.

5.2 Aggregate results with multiple drift points

As previously mentioned in Chapter 2, the DDE lift curve trend characteristic should be considered as an indicator of the connection between the drifting probability of features and feature-wise correlation. An upward trending curve is a proxy for a strong connection, while a flat one is a sign of the lack of the co-drifting phenomena (i.e., lack of the DDE effect).

Figures 3 and 4 present the results of DDE probability curves with more drift points, where the average results are the dark lines in the middle, and the light areas show the variance of all curves belonging to different drift points. Results of two high-dimensional datasets and two low-dimensional datasets with natural and artificially induced drift points based on the Jensen–Shannon (JS) detector are shown in Fig. 3. While Fig. 3 contains the comparison between the curves on four different datasets by the same detector (Jensen–

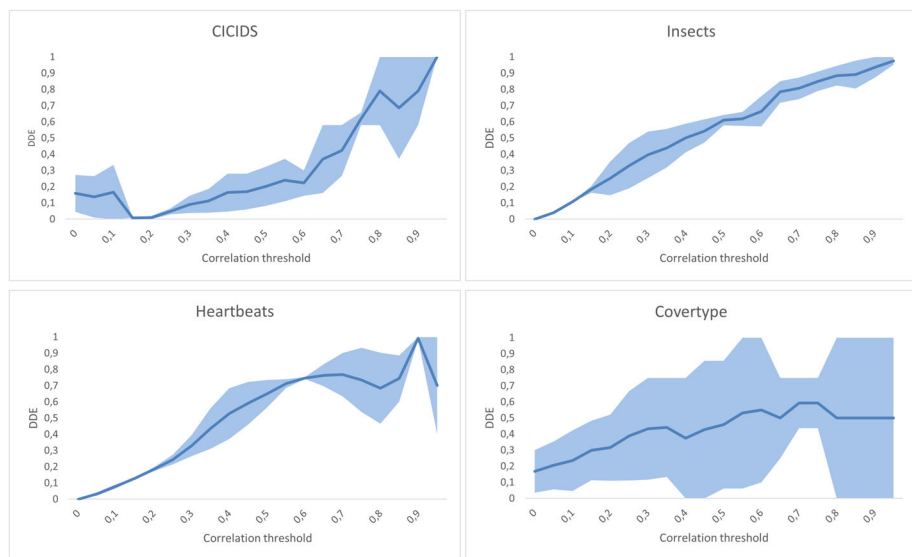


Fig. 3 Upward trending curves signal the Domino drift effect on all datasets (with Jensen–Shannon), except the Coverttype dataset, where the drift is artificially induced

Shannon), Fig. 4 shows the comparison between the curves of different detectors (Jensen–Shannon, Wasserstein, Kolmogorov–Smirnov, ADWIN) on the same dataset (Insects). As pictured in these figures, all three datasets containing real drifts present the DDE phenomena via upward trending curves, while the Coverttype shows a flattening lift. As mentioned earlier, Coverttype contains artificially induced drifts by exchanging feature values; therefore, as expected a flat curve indicates a non-existent DDE effect (with the high variance of DDE likelihood values between drift points).

As a counterexample, we introduced an artificial drift point into the Insects dataset by selecting a specific part of the series, where no natural drift occurrence was detected. As a next step, we exchanged the values of 60 randomly chosen features in the analysis window (and we do not modify the reference window). Modifying the value range and distribution of the features resulted in an artificially induced drift point (between the two windows) without respect to any real correlation. The consequence of this artificially induced drift point was a reverting and then flattening lift curve (as a clear sign of no real correlation drift connection) as can be seen in Fig. 5.

5.3 Evaluation of the prediction of new feature drifts

The DDE method—as described in Chapter 4.3—can be applied to predict drift behavior for each feature in future data streams by monitoring only a subset of a dataset. The evaluation of the method can be interpreted as the evaluation of a binary classification task, during which we want to determine whether each feature will drift or not. The application of the method is real importance in the case of high-dimension datasets, where significant resource savings are possible by ignoring the monitoring of lots of features. For this reason, the prediction was performed for only the two high-dimensional, Insects and Heartbeats, datasets. Two figures

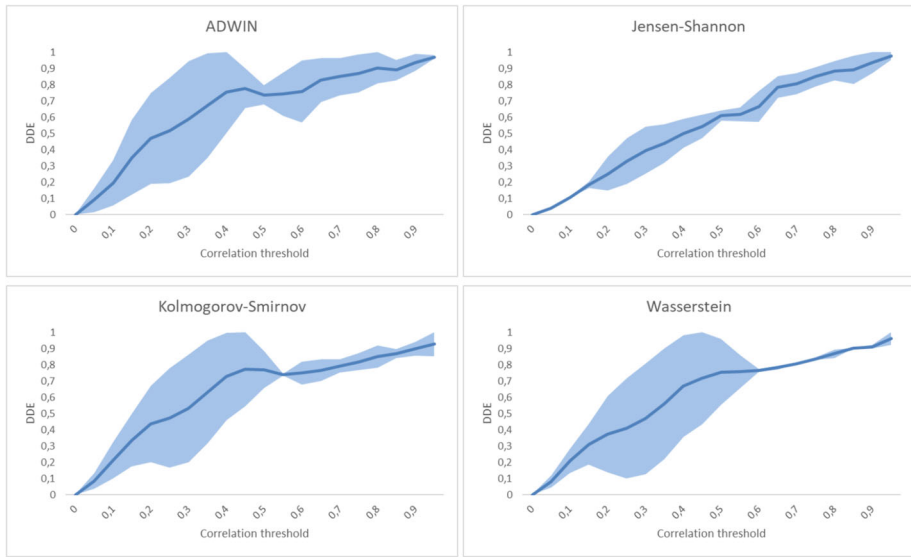


Fig. 4 Upward trending drift curves signal the Domino drift effect based on the four different detector outputs on the Insects dataset

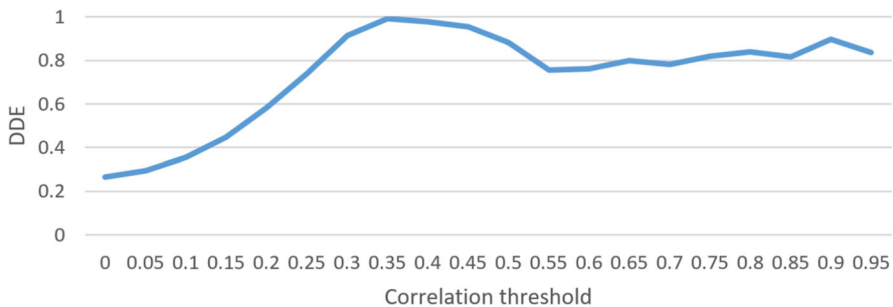


Fig. 5 Reverting and later flattening curve signals the lack of DDE effect for artificial drift induced to the Insects dataset

in Appendix present two ROC curve examples of predictions for the 2nd drift point of these datasets, where the size of S_{check} was 50.

During the evaluations, the size of S_{check} sets varied between $1 \dots m$ (number of all features). The runs were performed several times (10 folds); in each fold, our algorithm randomly selects the monitored features and averages the results. To evaluate the binary classification, the $F1$ -score and AUC metrics were measured, the values of which were calculated from the average values of the 10 unique folds (Figs. 6, 7, 8, 9 show the results).

As can be seen in Figs. 6, 7, 8, 9, fitting a DDE function on the 1st reference drift to predict the drifting features during the 2nd occurrence performs accurately for both datasets. Furthermore, the increasing number of features to be added to S_{check} not only increases the performance of the classifier but reduces the standard deviation of estimations also, as expected.

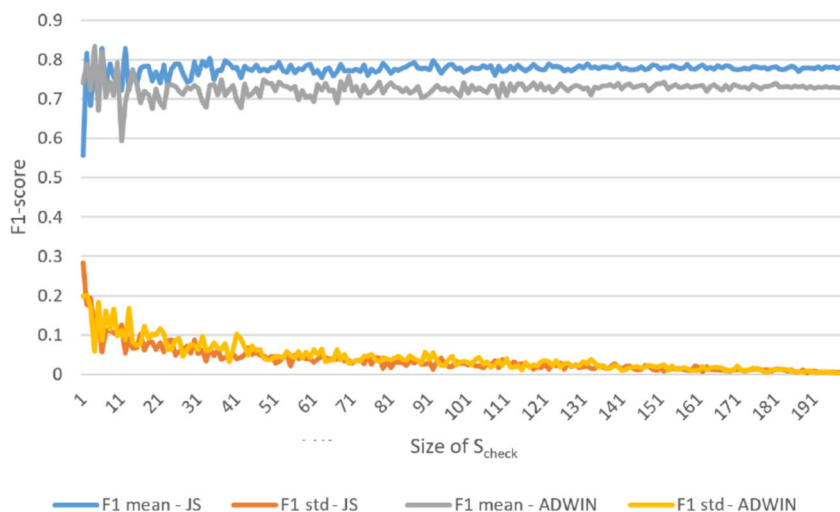


Fig. 6 F1-score-based performance of the 2nd drift point prediction varying the size of S_{check} in the Insects dataset

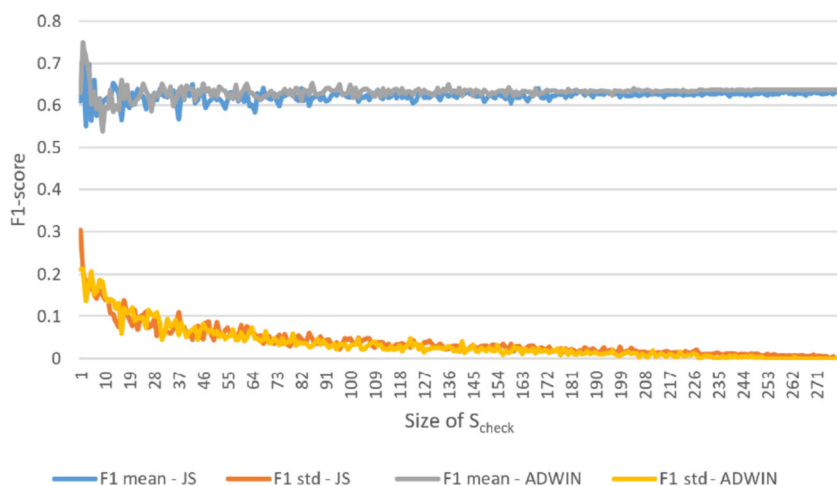


Fig. 7 F1-score-based performance of the 2nd drift point prediction varying the size of S_{check} in the Heartbeats dataset

5.4 Experiments on the effects of performance as a function of the number of monitored features

We conducted experiments to analyze the effects of performance as a function of the number of monitored features. This analysis is interpreted as the resources needed for drift detection, considering that the drift detection of a detector for each feature is of equal length. Our main hypothesis that utilizing the Domino drift effect significantly reduces the number of features needed to be monitored to detect the same number of drifts.

Tables 4 and 5 show the results of drift detection performance on the Heartbeats (also visualized in Fig. 10) and Insects high-dimensional datasets as a function of the computational

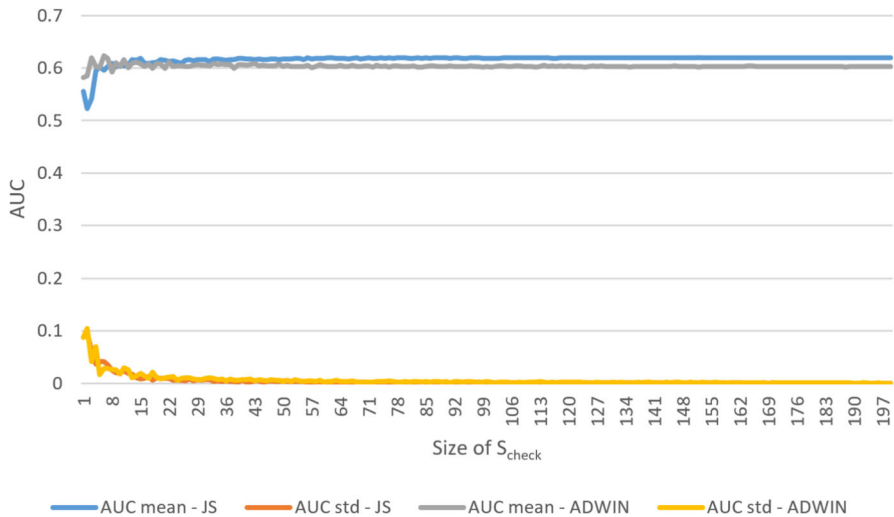


Fig. 8 AUC-score-based performance of the 2nd drift point prediction varying the size of S_{check} in the Insects dataset

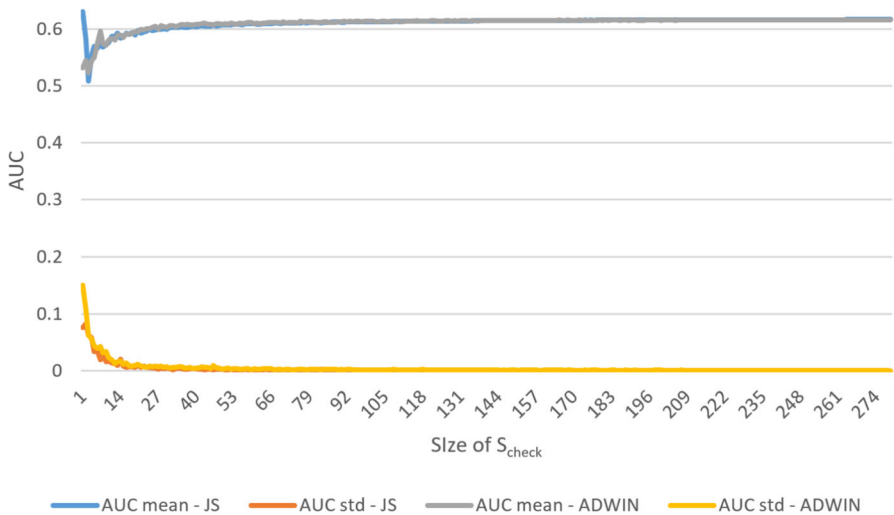


Fig. 9 AUC-score-based performance of the 2nd drift point prediction varying the size of S_{check} in the Heartbeats dataset

resources (number of features monitored with four different detectors). For both datasets, the resource requirements can be significantly decreased to achieve well-performing drift detection across the entire feature space (the larger difference can be seen in the right direction of the diagram in Fig. 10), purely based on the co-drifting phenomenon of the features described by the DDE method.

In Table 6, the results for the two lower-dimensional datasets are presented. In these cases, the reduction in the number of monitored features was limited by the lower number of dimensions themselves.

Table 4 Performance measures as a function of monitored features—Heartbeats dataset

Resources (%)		No. Monitored features					
		280 100%	140 50%	70 25%	35 13%	17 6%	8 3%
DDE-ADWIN	Recall	1.0000	0.9992	0.9930	0.9672	0.9175	0.8436
	Precision	0.7871	0.5297	0.3964	0.3367	0.3079	0.2951
	<i>F</i> 1	0.8808	0.6924	0.5666	0.4996	0.4610	0.4373
DDE-KS	Recall	0.9878	0.8950	0.8150	0.8004	0.7055	0.6924
	Precision	0.7623	0.6000	0.5147	0.4609	0.4170	0.3704
	<i>F</i> 1	0.8605	0.7184	0.6309	0.5850	0.5242	0.4826
DDE-Wass	Recall	0.9911	0.9839	0.9759	0.9785	0.9491	0.8907
	Precision	0.7787	0.6674	0.5916	0.5614	0.5354	0.5318
	<i>F</i> 1	0.8721	0.7953	0.7366	0.7134	0.6846	0.6660
DDE-JS	Recall	1.0000	1.0000	1.0000	1.0000	1.0000	0.7000
	Precision	0.7845	0.8061	0.7774	0.7975	0.7644	0.6750
	<i>F</i> 1	0.8792	0.8922	0.8724	0.8760	0.8548	0.6833

Table 5 Performance measures as a function of monitored features—Insects dataset

Resources (%)		No. Monitored features					
		200 100%	100 50%	50 25%	25 13%	12 6%	6 3%
DDE-ADWIN	Recall	0.9800	0.9963	0.9642	0.8608	0.7254	0.5586
	Precision	0.8680	0.5712	0.4261	0.3634	0.3232	0.2952
	<i>F</i> 1	0.9206	0.7261	0.5910	0.5110	0.4471	0.3863
DDE-KS	Recall	1.0000	1.0000	1.0000	0.9985	0.9455	0.7769
	Precision	0.7576	0.5955	0.4645	0.4007	0.3571	0.3429
	<i>F</i> 1	0.8621	0.7465	0.6343	0.5719	0.5184	0.4758
DDE-Wass	Recall	1.0000	1.0000	1.0000	0.9984	0.8957	0.8654
	Precision	0.8730	0.5945	0.4580	0.3854	0.3348	0.3374
	<i>F</i> 1	0.9322	0.7457	0.6283	0.5561	0.4874	0.4855
DDE-JS	Recall	0.9091	0.9229	0.9091	0.9213	0.8060	0.9000
	Precision	0.7576	0.7634	0.7867	0.7506	0.6615	0.5333
	<i>F</i> 1	0.8264	0.8346	0.8407	0.8189	0.7013	0.5567

In Fig. 11, the overall performance measured by the *F*1-score for all datasets is shown. Here, the horizontal axis shows not the absolute numbers of monitored features but their relative proportions. The performance curves were similar in three natural drift datasets, but in the artificially induced Covertype dataset it decreased by a much larger amount.

We compared our solution with three other methods, two baseline methods (Full and Reduced monitoring methods) and an additional simple (so-called reduced random) method; they are described below.

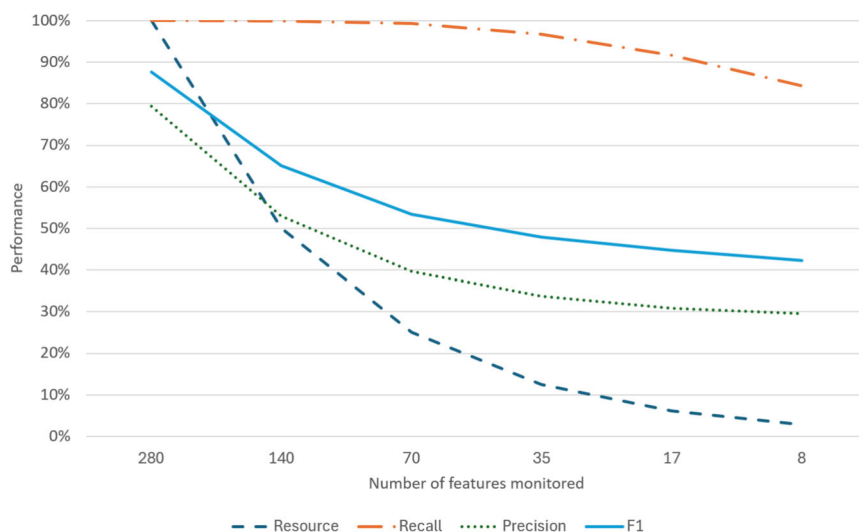


Fig. 10 Resources and goodness of drift detection on the Heartbeats dataset

Table 6 Drift detection performance on lower-dimensional datasets

Dataset	No. Monitored features	Resource (%)	Recall	Precision	F1
CICIDS	19	100.00	1.0000	0.9000	0.9400
	9	47.37	0.7758	0.7148	0.7122
COVERTYPE	16	100.00	1.0000	1.0000	1.0000
	7	43.75	0.6177	0.5215	0.5492

Full monitoring method: it monitors all features during the whole time in every time step; obviously, this is a resource wasting solution, because the drift detectors are called far too often. This baseline method is only basic from the resource point of view, but from the goodness viewpoint it is a very good solution that cannot be expected to be better.

Reduced monitoring method: this solution investigates only a subset of all features during the whole time. The method postpones the possibility of finding a drift among the remaining features. Thus, this baseline represents a missed opportunity to find additional drifts besides the drifts among the monitored features (so this does not use DDE); thus, this baseline method is basic from the goodness point of view.

Reduced random method: this is similar to our DDE method but instead of drift prediction to rest features, this basic method predicts randomly. The solution investigates only a subset of all features during the whole time, and if a drift occurs among the monitored features, then it predicts additional drifts among the unmonitored features. This prediction is based on the global probability of a drift, where this probability is estimated by the ratio of the number of drifted features and the number of unmonitored features. Besides the global probability, there is no other information in this method, so the features are randomly predicted for drift.

The results of the comparison are shown in Tables 7 and 8 for Heartbeats and Insects datasets, respectively. The best recall and F1-scores are achieved by our DDE method (in almost all cases), and the highest precision values belong to the reduced monitoring method,

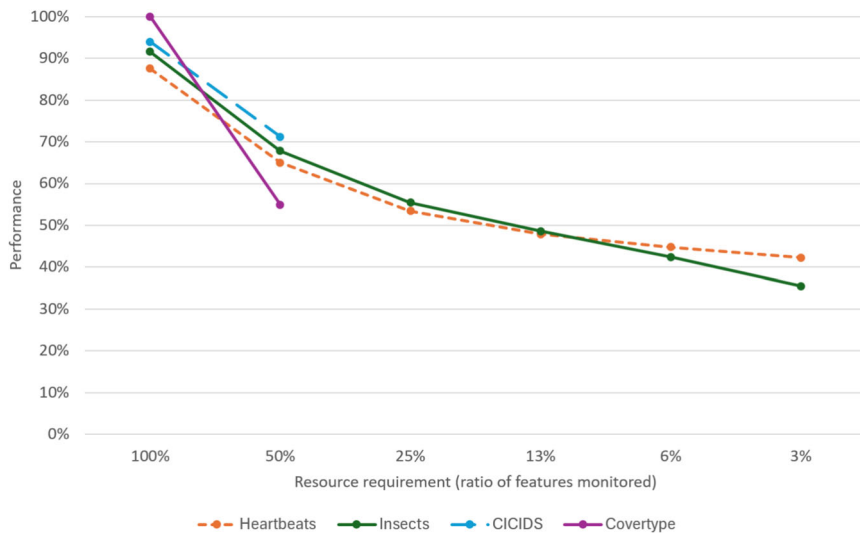


Fig. 11 Resources and $F1$ -scores of drift detection on all datasets

Table 7 Comparison of DDE method with three other methods on the Heartbeats dataset

		Resources (%)	No. Monitored features					
			280 100%	140 50%	70 25%	35 13%	17 6%	8 3%
DDE	Recall	–	–	0.9992	0.9930	0.9672	0.9175	0.8436
	Precision	–	–	0.5297	0.3964	0.3367	0.3079	0.2951
	$F1$	–	–	0.6924	0.5666	0.4996	0.4610	0.4373
Reduced monitoring	Recall	–	–	0.7000	0.3500	0.1750	0.0850	0.0400
	Precision	–	–	0.8000	0.7800	0.8000	0.8100	0.7800
	$F1$	–	–	0.7467	0.4832	0.2872	0.1539	0.0761
Reduced random	Recall	–	–	0.4119	0.2594	0.2000	0.1630	0.1393
	Precision	–	–	0.6179	0.4548	0.3750	0.3151	0.2747
	$F1$	–	–	0.4943	0.3304	0.2609	0.2148	0.1849
Full monitoring	Recall	–	1.0000	–	–	–	–	–
	Precision	–	0.7940	–	–	–	–	–
	$F1$	–	0.8852	–	–	–	–	–

The best (largest) recall, precision, and $F1$ values are denoted by bolds

but this was only due to the fact that the set of unmonitored features was not included in the decisions, so the number of errors that could be made here was in principle zero.

Tables 9 and 10 show the running times of the DDE and other three methods for Heartbeats and Insects datasets, respectively. In the full method, there is no initial value, but in the others the end users can choose a different number of monitored features according to their own available resources. This flexibility is a big advantage in our method. Despite the fact that two other methods have this advantage, the DDE method outperforms the others.

Table 8 Comparison of DDE method with three other methods on the Insects dataset

		Resources (%)	No. Monitored features					
			200 100%	100 50%	50 25%	25 13%	12 6%	6 3%
DDE	Recall	–		0.9963	0.9642	0.8608	0.7254	0.5586
	Precision	–		0.5712	0.4261	0.3634	0.3232	0.2952
	F1	–		0.7261	0.5910	0.5110	0.4471	0.3863
Reduced monitoring	Recall	–		0.5000	0.2500	0.1250	0.0570	0.0285
	Precision	–		0.8700	0.8700	0.9100	0.8000	0.8900
	F1	–		0.6350	0.3884	0.2198	0.1064	0.0552
Reduced random	Recall	–		0.4333	0.2626	0.1928	0.1646	0.1548
	Precision	–		0.6500	0.4583	0.3602	0.3204	0.3041
	F1	–		0.5200	0.3339	0.2512	0.2175	0.2052
Full monitoring	Recall		0.9800	–	–	–	–	–
	Precision		0.8680	–	–	–	–	–
	F1		0.9206	–	–	–	–	–

The best (largest) recall, precision, and F1 values are denoted by bolds

Table 9 Runtimes of the methods on the Heartbeats dataset (in secundum)

		No. Monitored features					
		280	140	70	35	17	8
ADWIN	Full	714	–	–	–	–	–
	DDE	–	357.3	178.9	89.7	43.8	20.9
	Reduced monitoring	–	357.0	178.5	89.3	43.4	20.4
	Reduced random	–	357.1	178.7	89.5	43.6	20.7
KS	Full	238	–	–	–	–	–
	DDE	–	119.3	59.9	30.2	14.9	7.3
	Reduced monitoring	–	119.0	59.5	29.8	14.5	6.8
	Reduced random	–	119.1	59.7	30.0	14.7	7.1
Wass	Full	166.6	–	–	–	–	–
	DDE	–	83.6	42.1	21.3	10.6	5.2
	Reduced monitoring	–	83.3	41.7	20.8	10.1	4.8
	Reduced random	–	83.4	41.9	21.1	10.4	5.0
JS	Full	126.14	–	–	–	–	–
	DDE	–	63.4	31.9	16.2	8.1	4.1
	Reduced monitoring	–	63.1	31.5	15.8	7.7	3.6
	Reduced random	–	63.2	31.7	16.0	7.9	3.9

6 Conclusion

Data drift detection is a challenge in itself during the development and operation of machine learning models, but if the detection takes place even in high-dimensional data sets, it causes extra difficulties for researchers. In this research, we have developed a method using feature

Table 10 Runtimes of the methods on the Insects dataset (in secundum)

		No. Monitored features					
		200	100	50	25	12	6
ADWIN	Full	510	–	–	–	–	–
	DDE	–	255.3	127.9	64.1	31.0	15.7
	Reduced monitoring	–	255.0	127.5	63.8	30.6	15.3
	Reduced random	–	255.1	127.7	63.9	30.8	15.5
KS	Full	170	–	–	–	–	–
	DDE	–	85.3	42.9	21.6	10.6	5.5
	Reduced monitoring	–	85.0	42.5	21.3	10.2	5.1
	Reduced random	–	85.1	127.7	63.9	30.8	15.5
Wass	Full	119	–	–	–	–	–
	DDE	–	59.8	30.1	15.3	7.5	4.0
	Reduced monitoring	–	59.5	29.8	14.9	7.1	3.6
	Reduced random	–	59.6	29.9	15.1	7.3	3.8
JS	Full	90.1	–	–	–	–	–
	DDE	–	45.4	22.9	11.6	5.8	3.1
	Reduced monitoring	–	45.1	22.5	11.3	5.4	2.7
	Reduced random	–	45.2	22.7	11.4	5.6	2.9

correlation as a drift proxy, which can be used to assign probability values to the expected drift of the remaining features by analyzing a low-dimensional subset.

We introduced a new concept, the Domino drift effect (DDE) for the co-drifting phenomenon, which is based on the assumption that the strong correlation between features is proportional to the probability of occurrence of the double drift phenomenon (both members of the feature pair drift). Our theoretical model takes advantage of this drift effect. Then we can make predictions about the drifting chances of one of the feature members of the pair without actually performing a drift detection on it, purely based on the drift behavior of the other feature.

We also proved the correctness and showed the practical application of our theory on four real-world datasets with low and high dimensions, containing artificially induced and naturally occurring data drifts also. The measurements were divided into two parts: we showed that in the case of natural drifts (Insects, Heartbeats, and CICIDS datasets), the DDE curve shows an increasing trend between the magnitude of the correlation and the DDE likelihood, but at the same time in the case of synthetically induced drifts (Coverttype), this relationship does not hold above, a flat trend can be observed.

Although the necessity of detecting at least one initial drift is a limitation of our approach, the curve and subsequent predictions cannot be obtained without this initialization phase. The advantage of our solution is that it does not need to use drift detectors on all features after the initialization phase, but only on a subset. Even if our solution cannot reach the same goodness as the drift detectors on the unmonitored subset, it can achieve relatively good predictions using the DDE based on the co-drifting phenomenon.

Finally, we illustrated the practical importance of the method with drift prediction tasks performed on two high-dimensional data sets: we used models fitted to previous drifts to tell features whether they would drift or not during subsequent streams. The practical application

of our theory performs well even with a small number of monitored features, but we also presented that by expanding the monitored feature set, the variance of the performance decreases.

Appendix A Appendix

See Fig. 12.

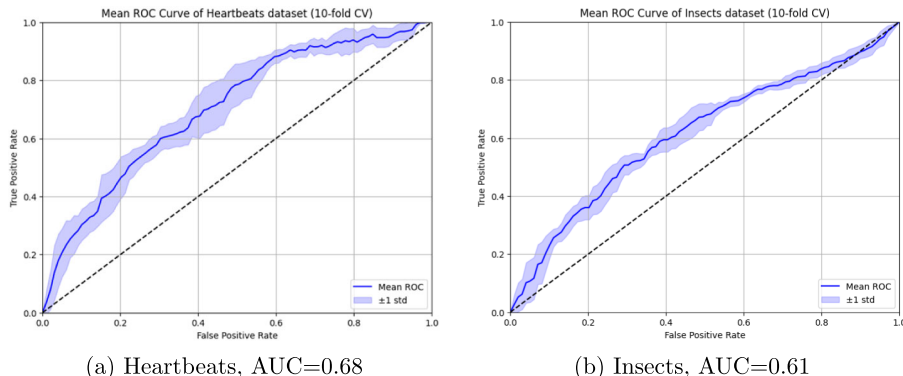


Fig. 12 Aggregated ROC curves for $|S_{check}| = 50$ based 10-fold cross-validation predictions at the 2nd drift point of the Heartbeats and Insects datasets

Acknowledgements Project No. KDP-IKT-2023-900-I1-00000957/0000003 has been implemented with the support provided by the Ministry of Culture and Innovation of Hungary from the National Research, Development and Innovation Fund, financed under the C2299763 funding scheme.

Author Contributions All authors contributed to the study conception and design. Gábor Szűcs planned the majority of the theoretical part and supervised the research. Marcell Németh developed and implemented the majority of the experimental part with data preparation and analysis. The authors contributed equally to this work. All authors read and approved the final manuscript.

Funding Open access funding provided by Budapest University of Technology and Economics.

Data availability There are no new datasets in this paper. The used datasets (Coverttype, CICIDS, Insects, Heartbeats) are available from citations written in the References.

Declarations

Conflict of interest The authors, Gábor Szűcs and Marcell Németh declare that they have no conflict of interest.

Ethical approval Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory

regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Guo H, Li H, Sun N, Ren Q, Zhang A, Wang W (2022) Concept drift detection and accelerated convergence of online learning. *Knowl Inf Syst* 65(3):1005–1043. <https://doi.org/10.1007/s10115-022-01790-6>
- Németh M, Szűcs G (2022) Split feature space ensemble method using deep reinforcement learning for algorithmic trading. In: Proceedings of the 2022 8th international conference on computer technology applications, pp 188–194. <https://doi.org/10.1145/3543712.3543722>
- Parasteh S, Sadaoui S (2024) A probabilistic approach for detecting real concept drift. In: ICAART (2), pp 301–311. <https://doi.org/10.5220/0012378800003636>
- Agrahari S, Srivastava S, Singh AK (2024) Review on novelty detection in the non-stationary environment. *Knowl Inf Syst* 66(3):1549–1574. <https://doi.org/10.1007/s10115-023-02018-x>
- Paldino GM, Lebiclot B, Le Borgne Y-A, Siblini W, Oblé F, Boracchi G, Bontempi G (2022) The role of diversity and ensemble learning in credit card fraud detection. *Adv Data Anal Classif*. <https://doi.org/10.1007/s11634-022-00515-5>
- Olorunnimbe M, Viktor H, Paquet E (2018) Dynamic adaptation of online ensembles for drifting data streams. *J Intell Inf Syst* 50:291–313. <https://doi.org/10.1007/s10844-017-0460-9>
- Lu J, Liu A, Dong F, Gu F, Gama J, Zhang G (2019) Learning under concept drift: a review. *IEEE Trans Knowl Data Eng* 31(12):2346–2363. <https://doi.org/10.1109/TKDE.2018.2876857>
- Kuncheva LI, Faithfull WJ (2014) PCA feature extraction for change detection in multidimensional unlabeled data. *IEEE Trans Neural Netw Learn Syst* 25(1):69–80. <https://doi.org/10.1109/TNNLS.2013.2248094>
- Kulinski S, Bagchi S, Inouye DI (2020) Feature shift detection: localizing which features have shifted via conditional distribution tests. *Adv Neural Inf Process Syst* 33:19523–19533
- Kuncheva LI (2013) Change detection in streaming multivariate data using likelihood detectors. *IEEE Trans Knowl Data Eng* 25(5):1175–1180. <https://doi.org/10.1109/TKDE.2011.226>
- Sethi TS, Kantardzic M (2017) On the reliable detection of concept drift from streaming unlabeled data. *Expert Syst Appl* 82:77–99. <https://doi.org/10.1016/j.eswa.2017.04.008>
- Micevska S, Awad A, Sakr S (2021) SDDM: an interpretable statistical concept drift detection method for data streams. *J Intell Inf Syst* 56:459–484. <https://doi.org/10.1007/s10844-020-00634-5>
- Coil C, Corizzo R (2024) Assessing distance measures for change point detection in continual learning scenarios. In: International symposium on methodologies for intelligent systems, pp 260–270. https://doi.org/10.1007/978-3-031-62700-2_23. Springer
- Bandara WGC, Patel VM (2022) A transformer-based siamese network for change detection. In: IGARSS 2022-2022 IEEE international geoscience and remote sensing symposium, pp 207–210. <https://doi.org/10.1109/IGARSS46834.2022.9883686>. IEEE
- Gözüağık Ö, Büyükcakır A, Bonab H, Can F (2019) Unsupervised concept drift detection with a discriminative classifier. In: Proceedings of the 28th ACM international conference on information and knowledge management, pp 2365–2368
- Shen P, Ming Y, Li H, Gao J, Zhang W (2022) Unsupervised concept drift detectors: a survey. In: The international conference on natural computation, fuzzy systems and knowledge discovery, pp 1117–1124. Springer
- Gemaque RN, Costa AFJ, Giusti R, Dos Santos EM (2020) An overview of unsupervised drift detection methods. *Wiley Interdiscip Rev Data Min Knowl Discov* 10(6):1381
- Cerqueira V, Gomes HM, Bifet A (2020) Unsupervised concept drift detection using a student–teacher approach. In: Discovery science: 23rd international conference, DS 2020, Thessaloniki, Greece, October 19–21, 2020, Proceedings 23, pp 190–204. Springer
- Ali Abdu NA, Basulaim KO (2024) Machine learning in concept drift detection using statistical measures. *Int J Comput Appl* 46(5):281–291. <https://doi.org/10.1080/1206212X.2023.2289706>
- Bayram F, Ahmed BS, Kassler A (2022) From concept drift to model degradation: an overview on performance-aware drift detectors. *Knowl-Based Syst* 245:108632. <https://doi.org/10.1016/j.knosys.2022.108632>
- Salem O, Naït-Abdesselam F, Mehaoua A (2012) Anomaly detection in network traffic using Jensen–Shannon divergence. In: 2012 IEEE international conference on communications (ICC), pp 5200–5204. <https://doi.org/10.1109/ICC.2012.6364602>

22. Bifet A, Gavalda R (2007) Learning from time-changing data with adaptive windowing. In: Proceedings of the 2007 SIAM international conference on data mining, pp 443–448. SIAM
23. Viehmann T (2021) Partial wasserstein and maximum mean discrepancy distances for bridging the gap between outlier detection and drift detection. arXiv preprint [arXiv:2106.12893](https://arxiv.org/abs/2106.12893)
24. Porwik P, Dadzie BM (2022) Detection of data drift in a two-dimensional stream using the Kolmogorov–Smirnov test. *Procedia Comput Sci* 207:168–175
25. Korycki L, Krawczyk B (2019) Unsupervised drift detector ensembles for data stream mining. In: 2019 IEEE international conference on data science and advanced analytics (DSAA), pp 317–325 IEEE
26. Gonçalves PM, de Carvalho Santos SGT, Barros RSM, Vieira DCL (2014) A comparative study on concept drift detectors. *Expert Syst Appl* 41(18):8144–8156. <https://doi.org/10.1016/j.eswa.2014.07.019>
27. Yang W, Su R, Cheng Y, Guo J (2022) A concept drift detection approach based on Jensen-Shannon divergence for network traffic classification. In: Proceedings of the 2022 5th international conference on artificial intelligence and pattern recognition, pp 982–987
28. Fan Q, Liu C, Zhao Y, Li Y (2022) Unsupervised online concept drift detection based on divergence and EWMA. In: Asia-Pacific Web (APWeb) and web-age information management (WAIM) joint international conference on web and big data, pp 121–134. Springer
29. Wang Z, Wang W (2020) Concept drift detection based on kolmogorov–smirnov test. In: Artificial intelligence in China: proceedings of the international conference on artificial intelligence in China, pp 273–280. Springer
30. Dos Reis DM, Flach P, Matwin S, Batista G (2016) Fast unsupervised online drift detection using incremental Kolmogorov–Smirnov test. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 1545–1554
31. Jafseer K, Shailesh S, Sreekumar A (2023) Modeling concept drift detection as machine learning model using overlapping window and Kolmogorov–Smirnov test. In: Machine learning, image processing, network security and data sciences: select proceedings of 3rd international conference on MIND 2021, pp 113–129. Springer
32. Basterrech S, Platoš J, Rubino G, Woźniak M (2022) Experimental analysis on dissimilarity metrics and sudden concept drift detection. In: International conference on intelligent systems design and applications, pp 190–199. Springer
33. Tao Y, Li C, Liang Z, Yang H, Xu J (2019) Wasserstein distance learns domain invariant feature representations for drift compensation of e-nose. *Sensors* 19(17):3703
34. Huang DTJ, Koh YS, Dobbie G, Bifet A (2015) Drift detection using stream volatility. In: Machine learning and knowledge discovery in databases: European conference, ECML PKDD 2015, Porto, Portugal, September 7–11, 2015, Proceedings, Part I 15, pp 417–432. Springer
35. Alberge F, Feutry C, Duhamel P, Piantanida P (2019) Detecting covariate shift with black box predictors. In: 2019 26th international conference on telecommunications (ICT), pp 324–329. <https://doi.org/10.1109/ICT.2019.8798827>
36. Jang S, Park S, Lee I, Bastani O (2022) Sequential covariate shift detection using classifier two-sample tests. In: International conference on machine learning, pp 9845–9880. PMLR
37. Goldenberg I, Webb GI (2020) PCA-based drift and shift quantification framework for multidimensional data. *Knowl Inf Syst* 62(7):2835–2854. <https://doi.org/10.1007/s10115-020-01438-3>
38. Gulcan EB, Can F (2022) Unsupervised concept drift detection for multi-label data streams. *Artif Intell Rev* 56(3):2401–2434. <https://doi.org/10.1007/s10462-022-10232-2>
39. Adams JN, van Zelst SJ, Rose T, van der Aalst WMP (2023) Explainable concept drift in process mining. *Inf Syst* 114:102177. <https://doi.org/10.1016/j.is.2023.102177>
40. Blackard J, Dean D (1999) Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Comput Electron Agric* 24:131–151. [https://doi.org/10.1016/S0168-1699\(99\)00046-0](https://doi.org/10.1016/S0168-1699(99)00046-0)
41. Sharafaldin I, Lashkari AH, Ghorbani AA (2018) Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1:108–116. <https://doi.org/10.5220/0006639801080116>
42. Ulanova L, Begum N, Shokoohi-Yekta M, Keogh E, Clustering in the face of fast changing streams. In: Proceedings of the 2016 SIAM international conference on data mining, pp 1–9. <https://doi.org/10.1137/1.9781611974348.1>
43. Moody GB, Mark RG (2001) The impact of the MIT-BIH arrhythmia database. *IEEE Eng Med Biol Mag* 20(3):45–50. <https://doi.org/10.1109/51.932724>



Gábor Szűcs has received MSc in electrical engineering and PhD in computer science from the Budapest University of Technology and Economics (BME) in 1994 and in 2002, respectively. He is an associate professor at the Department of Telecommunications and Media Informatics of BME. His research areas are data science, artificial intelligence, deep learning, content-based image retrieval, and multimedia mining. The number of his publications is more than 100. He is the president of the Artificial Intelligence Section of HTE (Scientific Association for Infocommunications) and the leader of the research group DCLAB (Data Science and Content Technologies). He has earned János Bolyai Research Scholarship of the Hungarian Academy of Science some years ago.



Marcell Németh is a PhD student in computer science at the Budapest University of Technology, specializing in machine learning. His research focuses on time series classification, forecasting, and explainability, with an emphasis on handling temporal dynamic changes, feature extraction, and causal inference.